

S.S. GOVT. ARTS COLLEGE, TIRUTTANI – 631209

AI-AUGMENTED BACKEND DEVELOPMENT PROJECT

AI FitTrack

Personal Fitness Management System, Augmented with an AI Recommendation Layer

PROJECT TEAM

Name	Register No.	Role
Somu M P	212400292	Team Leader
Vedhanarayanan P	212400293	Team Member
Naveen Kumar S	212400285	Team Member
Perarasu S	212400287	Team Member
Pozhilan M	212400288	Team Member

TECHNOLOGY OVERVIEW

Field	Detail
Project Folder	AI-FITTRACK-API
Frontend	HTML + CSS + JavaScript
Backend	Node.js + Express.js
Database	MongoDB Atlas (Mongoose ODM)
Authentication	JWT + bcryptjs
AI Integration	Google Gemini API – fitness recommendation service

Submitted as part of the AI-Augmented Backend Development Project (SmartBridge – SkillWallet Program)

Table of Contents

- 1. Project Description
- 2. Scenario
- 3. System Requirements
- 4. Project Architecture
- 5. Database Design and Collection Relationships
- 6. Features
- 7. Roles and Responsibilities
- 8. User Flow
- 9. MVC Pattern
- 10. Project Setup and Configuration
- 11. Backend Development
- 12. AI Integration – Recommendation, Vector Search and Chat
- 13. Database Development
- 14. Project Execution
- 15. Testing Methodology and Results
- 16. Output Screenshots with Explanation
- 17. Additional AI, Vector Search and Admin Feature Screenshots
- 18. Demo Video and Source Code Links
- 19. Mandatory Functionalities Checklist
- 20. Conclusion
- 21. Future Enhancements

1. Project Description

AI FitTrack is an AI-Augmented backend web application developed using the MERN-style stack (MongoDB, Express.js, Node.js) with a browser-based HTML/CSS/JavaScript frontend, integrated with an Artificial Intelligence recommendation layer powered by the Google Gemini API. The system allows a registered user to record body-progress measurements, workouts and food/nutrition entries, view an automatically calculated BMI, and receive an AI-generated fitness recommendation based on that data. By combining a traditional JWT-secured REST backend with a generative-AI recommendation service, the platform delivers a single, organized dashboard that reduces manual fitness record-keeping and gives the user actionable, personalized guidance.

2. Scenario

Maintaining fitness information manually is difficult. A person trying to track body weight, workouts and daily food intake usually ends up with data scattered across notebooks, spreadsheets or several unrelated mobile apps. Without a single organized record, it becomes hard to see whether weight, BMI or workout consistency is actually improving over time, and there is no simple way to turn that raw data into a concrete next action.

A plain record-keeping system can store the numbers, but it cannot interpret them. Two users with the same BMI may need very different advice depending on their recent workout and food history, and writing fixed if-else rules for every possible combination does not scale. This is where Artificial Intelligence adds real value: a generative-AI model can be given the user's latest weight, height, BMI category and activity data as a prompt, and return a short, personalized recommendation in natural language instead of a generic static message.

AI FitTrack addresses this by combining a secure JWT-authenticated backend with the Google Gemini API. After a user logs in and adds progress, workout or food records, the backend assembles the latest BMI and activity summary into a prompt and sends it to the Gemini API; the response is stored in the recommendations collection and shown on the dashboard as the "AI Recommendation". Students, fitness beginners and anyone who wants a single organized place to track health data benefit from this approach — they get both a historical record and an AI-interpreted summary of what that record means, without needing a personal trainer to read the numbers for them.

3. System Requirements

3.1 Software Requirements

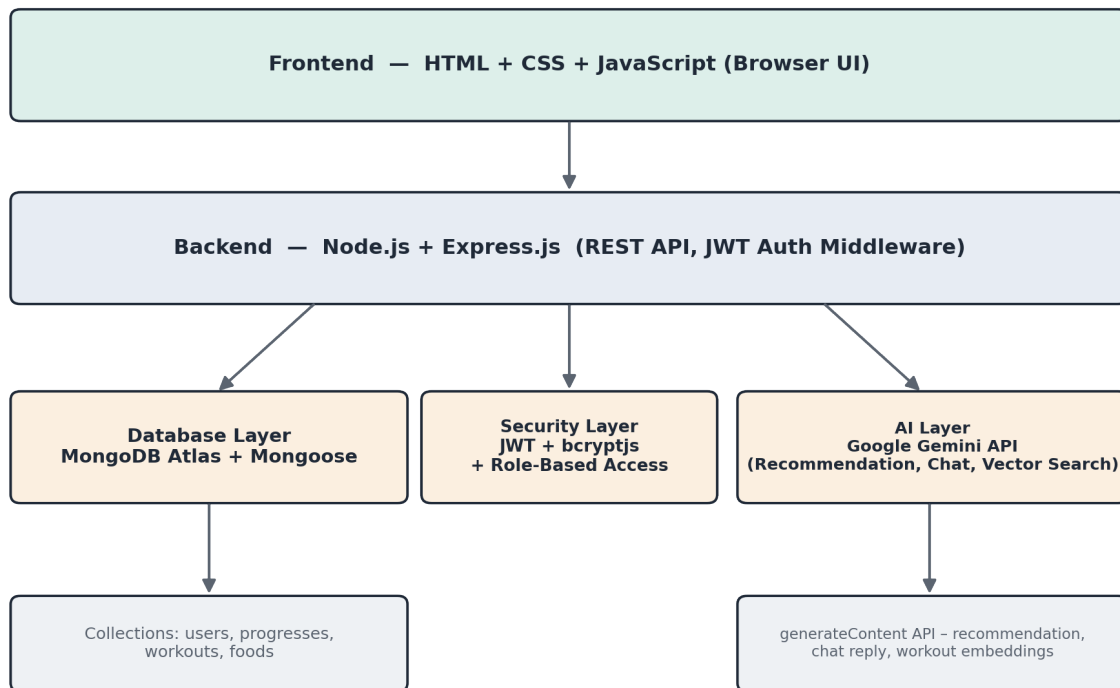
- Operating System – Windows 10/11
- Node.js (v18 or above) and npm (v9 or above)
- Express.js – backend REST framework
- MongoDB Atlas – cloud NoSQL database
- Mongoose – ODM for schema and queries
- Google Gemini API – AI recommendation service
- JWT (jsonwebtoken) and bcryptjs – authentication and password hashing
- Google Chrome / any modern browser
- Thunder Client (VS Code extension) – REST API testing
- Visual Studio Code – development environment
- Git and GitHub – version control

3.2 Hardware Requirements

- Processor: Intel Core i5 (8th Gen+) / AMD Ryzen 5 or above
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: Minimum 5 GB free space
- Internet connection – required for MongoDB Atlas and Gemini API calls
- Display: 1366 × 768 or above

4. Project Architecture

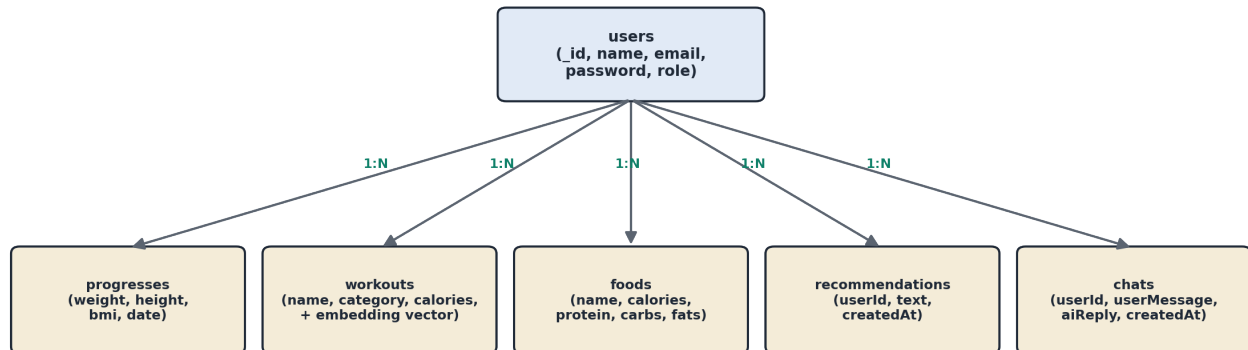
AI FitTrack follows a layered client-server architecture. The browser-based frontend sends HTTP requests to the Express backend; protected routes are guarded by JWT middleware; the backend reads and writes data through Mongoose to MongoDB Atlas, and calls the Gemini API to generate the AI recommendation text that is shown on the dashboard.



5. Database Design and Collection Relationships

MongoDB Atlas is used as the cloud database. The working Atlas cluster contains five collections: users, workouts, progresses, foods and recommendations. Every workout, progress and food record is linked back to the authenticated user who created it — a one-to-many relationship between users and each of the other collections.

One authenticated user owns many records across five collections



Collection	Purpose
users	Authentication and account information
progresses	Weight, height, BMI and date records
workouts	Workout name, category, duration, calories, date
foods	Food name, calories, protein, carbohydrates, fats, serving size
recommendations	AI-generated recommendation text linked to a user

6. Features

6.1 Core Backend Features

- User registration and login with password hashing (bcryptjs)
- JWT-based authentication and protected routes
- CRUD APIs for progress, workout and food records
- Automatic BMI calculation from weight and height
- Searchable workout and food listings
- Dashboard aggregation – total workouts, calories burned, total foods, latest progress

6.2 AI Features

- AI-Powered Fitness Recommendation – the backend sends the user's latest BMI, BMI category and recent activity summary to the Google Gemini API and stores/returns a short natural-language recommendation, displayed in the dashboard's "AI Recommendation" section.
- AI Chat – POST /api/ai/chat lets the user ask a free-text fitness question and receive a Gemini-generated conversational reply; each exchange is stored so chat history can be retrieved.
- Semantic / Vector Search – workout records are converted into embedding vectors (generateWorkoutEmbeddings.js) and POST /api/vector-search/search returns the workouts most similar to a free-text query, ranked by cosine-similarity score.

6.3 Access Control Feature

- Role-Based Access Control – user documents carry a role field; a dedicated admin middleware checks this role before allowing access to /api/admin/* routes (dashboard statistics, user list, role-verification test).

7. Roles and Responsibilities

The application supports two authenticated roles – User and Admin – distinguished by a role field on the user document and enforced through a dedicated admin middleware on every `/api/admin/*` route.

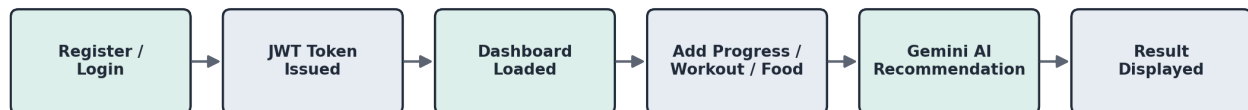
User

- Register and log in securely
- Add, view, edit and delete progress records
- Add, search, edit and delete workout records
- Add, search, edit and delete food records
- View dashboard statistics and the latest BMI category
- View the AI-generated fitness recommendation
- Ask fitness questions through the AI Chat and view chat history
- Search workouts semantically using the vector search endpoint

Admin

- Access platform-wide dashboard statistics (total users, workouts, foods, progress records) via GET `/api/admin/dashboard`
- View the list of all registered users via GET `/api/admin/users`
- Verify admin-role access via GET `/api/admin/test`

8. User Flow



- User registers or logs in
- Backend issues a JWT token, stored by the frontend
- Dashboard is loaded using the token (Authorization header)
- User adds a progress, workout or food record
- Backend recalculates dashboard totals and calls the Gemini API for a fresh recommendation
- Updated dashboard, BMI category and AI recommendation are displayed

9. MVC Pattern

The backend follows the MVC (Model–View–Controller) pattern, implemented with Mongoose models, Express route files and controller functions.

Model Layer (models/)

- User schema (includes role field)
- Progress schema
- Workout schema (includes embedding vector field)
- Food schema
- Recommendation schema
- Chat message schema

Controller Layer (controllers/)

- Authentication logic (register/login, password hashing, token issue)
- Progress, workout and food CRUD operations
- Dashboard aggregation logic
- aiController.js – Gemini API request/response handling for recommendations and chat
- vectorSearchController.js – embedding-based workout similarity search
- adminController.js – admin dashboard statistics, user list, role verification

Service Layer (services/)

- geminiService.js – wraps Gemini generateContent calls for recommendation and chat
- embeddingService.js – generates and compares workout embedding vectors
- aiRecommendation.js – builds the BMI/activity prompt and stores the response

Middleware Layer (middleware/)

- auth.js – verifies the JWT on protected routes
- admin.js – checks the authenticated user's role before allowing /api/admin/* access

View Layer / Routes (routes/)

- Auth routes – register, login
- Progress routes – CRUD
- Workout routes – CRUD + search
- Food routes – CRUD + search
- Dashboard route – aggregated statistics
- AI routes – recommendation, chat, chat history
- Vector search route – POST /api/vector-search/search
- Admin routes – dashboard, users, test

Advantages of MVC

- Separation of concerns
- Easier maintainability and testing
- Reusable controller logic
- Simple to extend with new modules

10. Project Setup and Configuration

10.1 Project Structure

Project folder: **AI-FITTRACK-API**

- server/ – backend application (app.js, server.js)
- server/models/ – Mongoose schemas
- server/controllers/ – request handlers
- server/routes/ – Express route definitions
- server/middleware/ – JWT auth middleware
- server/services/ – aiRecommendation.js, geminiService.js
- server/config/ – database connection
- server/client/ – frontend (index.html, style.css, app.js)

10.2 Server Setup

```
mkdir server
cd server
npm init -y
```

10.3 Install Backend Packages

```
npm install express mongoose cors dotenv bcryptjs jsonwebtoken
npm install @google/generative-ai
npm install nodemon --save-dev
```

11. Backend Development

The backend exposes REST endpoints for every module. All protected routes require a valid JWT sent as an Authorization: Bearer header.

Operation	HTTP Method	Purpose
Register	POST	Create user account
Login	POST	Authenticate user, issue JWT
Dashboard	GET	Retrieve aggregated dashboard information
Progress	GET / POST / PUT / DELETE	Manage progress (weight, height, BMI) records
Workouts	GET / POST / PUT / DELETE	Manage workout records
Workout Search	GET	Search workout records by name
Foods	GET / POST / PUT / DELETE	Manage food/nutrition records
AI Recommendation	POST	Generate a Gemini-based fitness recommendation
AI Chat	POST	Send a free-text question, receive a Gemini chat reply
AI Chat History	GET	Retrieve the user's previous chat messages
Vector Search	POST	Semantic search over workout embeddings, ranked by similarity
Admin Dashboard	GET	Platform-wide statistics (admin role required)
Admin Users	GET	List all registered users (admin role required)
Admin Test	GET	Verify the requester's token carries the admin role

- Express server configuration and CORS setup
- MongoDB/Mongoose connection (config/db.js)
- JWT authentication middleware on protected routes
- Progress, workout and food routes with validation
- Dashboard aggregation controller
- Gemini recommendation service
- Centralized error responses

12. AI Integration – Recommendation, Vector Search and Chat

12.1 AI Fitness Recommendation

Purpose

Convert the user's stored BMI/activity data into a short, personalized fitness recommendation shown on the dashboard, instead of a generic static message.

Request / Response Structure

The backend service (`geminiService.js`) builds a prompt containing the user's latest weight, height, BMI value and BMI category, and sends it to the Gemini `generateContent` endpoint using the API key configured in the environment file (`GEMINI_API_KEY`). The response text is extracted by `aiRecommendation.js`, stored in the `recommendations` collection against the user, and returned to the frontend as the "AI Recommendation" field.

12.2 AI Chat

Purpose

Let the user ask an open, free-text fitness question (for example, "I did 30 minutes of running today. Is this a good workout?") instead of only reading a fixed-format recommendation.

Request / Response Structure

`POST /api/ai/chat` accepts `{ message }` from the authenticated user, forwards it to the Gemini API through `geminiService.js`, and returns the generated reply. Every exchange (`userId`, `userMessage`, `aiReply`, `createdAt`) is saved, so `GET /api/ai/chat/history` can return the user's previous conversation.

12.3 Vector / Semantic Search

Purpose

Allow a workout to be found by meaning rather than an exact name match — a query such as "cardio running workout" should surface Running and Cycling entries, not just records that literally contain those words.

Implementation

`generateWorkoutEmbeddings.js` converts each workout's text fields into an embedding vector and stores it on the workout document. `POST /api/vector-search/search` embeds the incoming query the same way and ranks stored workouts by cosine-similarity score; `testVectorSearch.js` is a standalone script used to verify this ranking directly from the terminal.

12.4 Cost and Security Practices

- Each recommendation or chat request triggers one Gemini API call; a transient 503 ("model overloaded") response was observed during testing and the service recovered on retry — confirming a real external API call rather than a mocked response
- The Gemini API key is kept in a server-side `.env` file and is never exposed to the frontend
- Every AI, chat and vector-search route requires a valid JWT; admin-only routes additionally check the user's role through the admin middleware before responding

13. Database Development

The working MongoDB Atlas cluster (Cluster0 → aifittrack database) contains the five collections listed in Section 5. Screenshot 8 (Section 16) shows the actual collection sizes and document counts captured from Atlas Data Explorer.

- MongoDB collections – users, progresses, workouts, foods, recommendations
- One-to-many relationships from users to the other four collections
- Indexes on the users collection for unique email lookups
- Sample documents verified directly in Atlas Data Explorer

14. Project Execution

Step 1: Backend

```
cd server  
npm install
```

Step 2: Configure Environment Variables (.env)

```
PORT=5000  
MONGO_URI=mongodb+srv://:@cluster0.zipwpl.mongodb.net/aifittrack  
JWT_SECRET=your_secret_key  
GEMINI_API_KEY=your_google_gemini_api_key
```

Step 3: Run Backend

```
node server.js
```

Backend URL: <http://127.0.0.1:5000> | Frontend: [server/client/index.html](#) (served via a local server on port 5500)

15. Testing Methodology and Results

The backend APIs were tested using Thunder Client after starting the backend server and confirming the MongoDB Atlas connection.

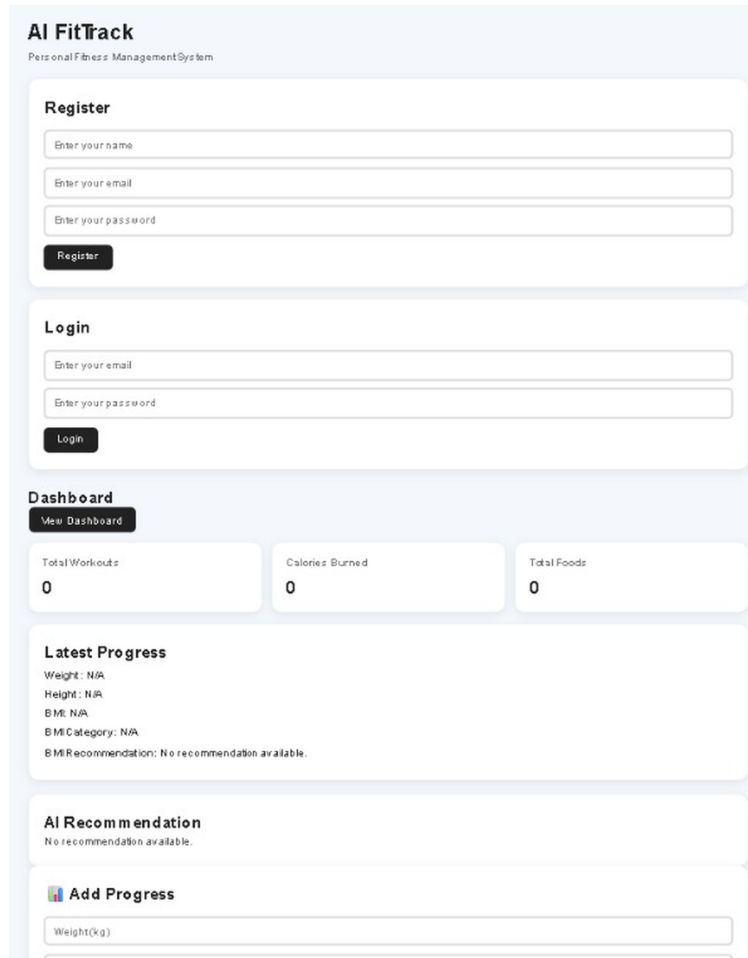
Test Area	Expected Result	Status
Registration	User account created	Completed
Login	Token returned, dashboard accessible	Completed
Progress	Record stored and BMI displayed	Completed
Workout CRUD	Create/read/update/delete works	Completed
Workout search	Matching records displayed	Completed
Food records	Food information displayed	Completed
MongoDB Atlas	Collections and documents visible	Completed
AI Recommendation	Gemini response stored and displayed	Completed
Dashboard	Counts and latest progress shown	Completed

16. Output Screenshots with Explanation

This section presents screenshots of the working application, the MongoDB Atlas database and the backend server, in the order the features are described above.

Screenshot 1

Frontend – AI FitTrack Main Page



AI FitTrack
Personal Fitness Management System

Register

Enter your name

Enter your email

Enter your password

Register

Login

Enter your email

Enter your password

Login

Dashboard

View Dashboard

Total Workouts
0

Calories Burned
0

Total Foods
0

Latest Progress

Weight: N/A
Height: N/A
BMI: N/A
BMI Category: N/A
BMI Recommendation: No recommendation available.

AI Recommendation

No recommendation available.

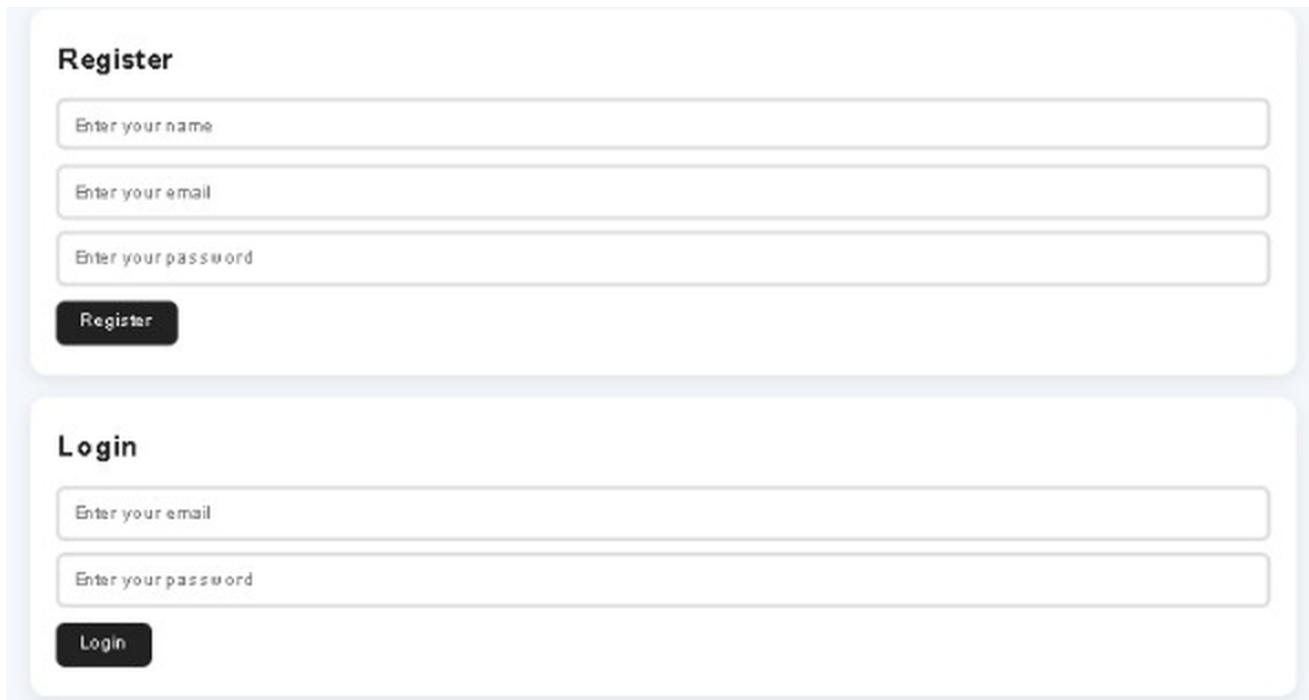
Add Progress

Weight(kg)

The landing page served from server/client/index.html, showing the Register, Login and Dashboard sections.

Screenshot 2

Registration and Login



The screenshot displays two distinct forms for user authentication. The top form, titled 'Register', contains three input fields: 'Enter your name', 'Enter your email', and 'Enter your password', followed by a dark 'Register' button. The bottom form, titled 'Login', contains two input fields: 'Enter your email' and 'Enter your password', followed by a dark 'Login' button. Both forms are set against a light blue background with rounded corners.

Register

Enter your name

Enter your email

Enter your password

Register

Login

Enter your email

Enter your password

Login

The Register and Login forms that call the `/api/auth/register` and `/api/auth/login` endpoints.

Screenshot 3

Login Success and Dashboard



The screenshot shows a web application interface. At the top, there is a 'Login' section with two input fields: the first contains the email 'somu123@gmail.com' and the second contains masked characters '.....'. Below these fields is a dark 'Login' button. Below the login section is a 'Dashboard' section. It features a dark 'View Dashboard' button. Underneath, there are three white cards displaying statistics: 'Total Workouts' with the value '6', 'Calories Burned' with the value '1150', and 'Total Foods' with the value '4'.

Category	Value
Total Workouts	6
Calories Burned	1150
Total Foods	4

After a successful login the JWT is stored and the dashboard loads Total Workouts, Calories Burned and Total Foods.

Screenshot 4

Add Progress – Automatic BMI

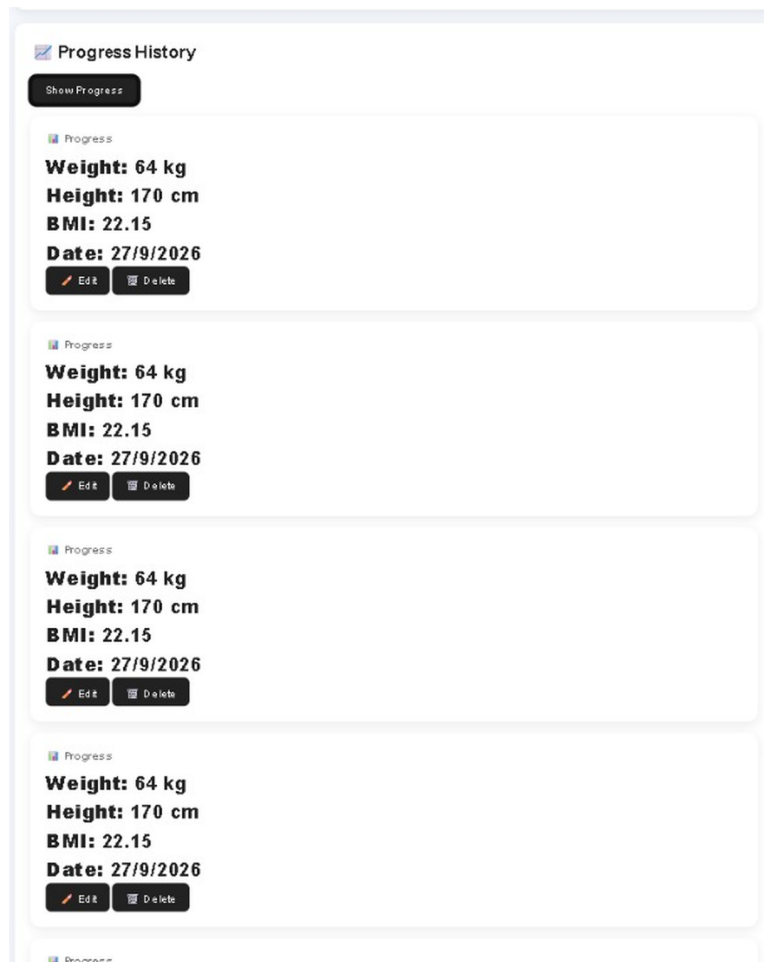


The screenshot shows a web form titled "Add Progress" with a small bar chart icon. It contains four input fields stacked vertically. The first field contains the number "64". The second field contains "170" and is highlighted with a thick black border. The third field contains "22.15". The fourth field contains the placeholder text "dd-mm-yyyy" and has a calendar icon on its right side. Below these fields is a dark button labeled "Add Progress".

Weight and height are entered; the BMI field is auto-calculated on the frontend before the record is saved.

Screenshot 5

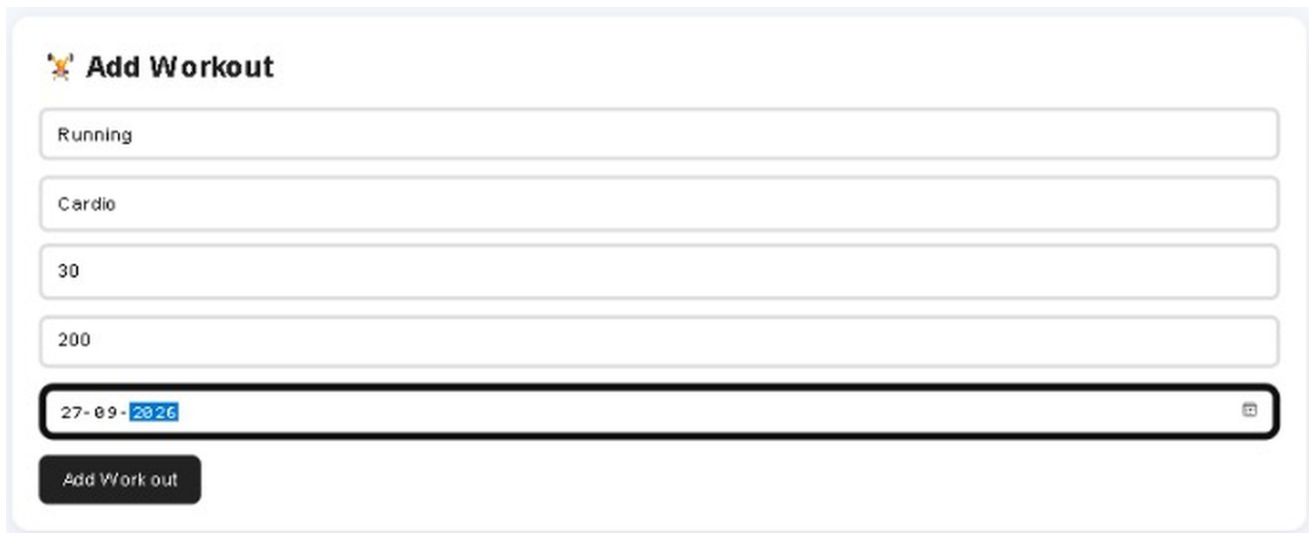
Progress History



Stored progress records with weight, height, BMI and date, each with Edit and Delete actions.

Screenshot 6

Add Workout



The screenshot shows a web form titled "Add Workout" with a small icon of a person running. The form contains four input fields: "Running", "Cardio", "30", and "200". Below these is a date and time field showing "27-09-2026" with a calendar icon on the right. At the bottom is a dark button labeled "Add Work out".

The Add Workout form capturing name, category, duration and calories burned.

Screenshot 7

Workout List and Search

 **MyWorkouts**

Search

Show AllWorkouts

Running

Category: Cardio
Duration: 30 minutes
Calories: 250
Date: 27/9/2026

Edit

Delete

Walking

Category: Cardio
Duration: 20 minutes
Calories: 100
Date: 27/9/2026

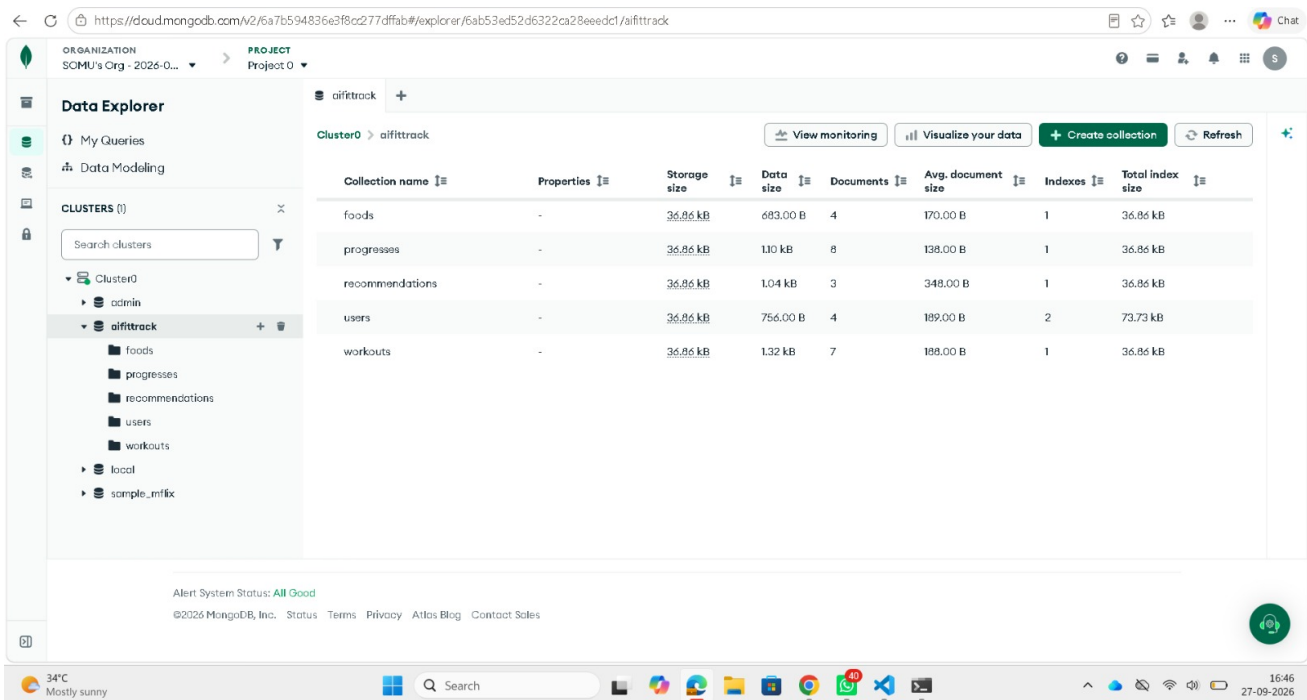
Edit

Delete

The workout list with the search box used to filter records by name.

Screenshot 8

MongoDB Atlas – FitTrack Collections



The screenshot shows the MongoDB Atlas Data Explorer interface for the 'aifittrack' database. The left sidebar displays the 'Data Explorer' section with 'My Queries' and 'Data Modeling' options. Under 'CLUSTERS (1)', the 'aifittrack' cluster is selected, showing a tree view of collections: 'foods', 'progresses', 'recommendations', 'users', and 'workouts'. The main panel displays a table of these collections with their properties, storage size, data size, document counts, average document size, number of indexes, and total index size.

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
foods	-	36.86 kB	683.00 B	4	170.00 B	1	36.86 kB
progresses	-	36.86 kB	110 kB	8	138.00 B	1	36.86 kB
recommendations	-	36.86 kB	1.04 kB	3	348.00 B	1	36.86 kB
users	-	36.86 kB	756.00 B	4	189.00 B	2	73.73 kB
workouts	-	36.86 kB	1.32 kB	7	188.00 B	1	36.86 kB

Alert System Status: All Good
 ©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

The aifittrack database in Atlas Data Explorer, showing the five live collections and document counts.

Screenshot 9

Add Food and Food List


Add Food

Add Food


Food List

Search
Show All Foods


Apple

Calories: 52
Protein: 0.3 g
Carbohydrates: 14 g
Fats: 0.2 g
Serving Size: 100g

Edit
Delete


banana

Calories: 52
Protein: 0.2 g
Carbohydrates: 55 g

The Add Food form and the resulting food list with calories, protein, carbohydrates and fats.

Screenshot 10

AI Recommendation

Latest Progress

Weight: 64 kg
Height: 170 cm
BM: 22.15
BM Category: Normal
BM Recommendation: Your BMI is in the normal range. Continue maintaining balanced nutrition and regular physical activity.

AI Recommendation

Your BMI is in the normal range. Maintain a balanced diet and regular exercise. Try to maintain a more consistent workout schedule during the week. Gradually increase your physical activity according to your fitness level.

The Latest Progress card and the Gemini-generated AI Recommendation text shown on the dashboard.

Screenshot 11

Final Dashboard – Results

Login

Login

Dashboard

View Dashboard

Total Workouts

6

Calories Burned

1150

Total Foods

4

Latest Progress

Weight: 64 kg

Height: 170 cm

BMI: 22.15

BMI Category: Normal

BMI Recommendation: Your BMI is in the normal range. Continue maintaining balanced nutrition and regular physical activity.

AI Recommendation

Your BMI is in the normal range. Maintain a balanced diet and regular exercise. Try to maintain a more consistent workout schedule during the week. Gradually increase your physical activity according to your fitness level.

Add Progress

64

170

22.15

dd-mm-yyyy

Add Progress

The dashboard after several records were added, showing updated totals, BMI category and AI recommendation.

Screenshot 12

Final Application – Complete View

AI FitTrack
Personalized fitness management system

Register

First Name:

Last Name:

Email:

Phone:

Register

Login

Email:

Password:

Login

Dashboard

Overview

Total Workouts: 5 | Current Distance: 990 | Total Steps: 4

Latest Progress

Workout: 5km
Progress: 100%
Date: 2024-10-26
Category: Running
Description: You did an intense 5km run today, maintaining a steady pace and pushing your physical ability.

AI Recommendation

You did an intense 5km run today, maintaining a steady pace and pushing your physical ability. To continue your progress, we recommend you try a 10km run next time.

Add Progress

Distance:

Date:

Category:

Add Progress

Progress History

View Progress

Click "View Progress" to view your progress.

Add Workout

Workout Name:

Category:

Distance (miles):

Calories Burned:

Date:

Add Workout

My Workouts

Get 10 workouts...

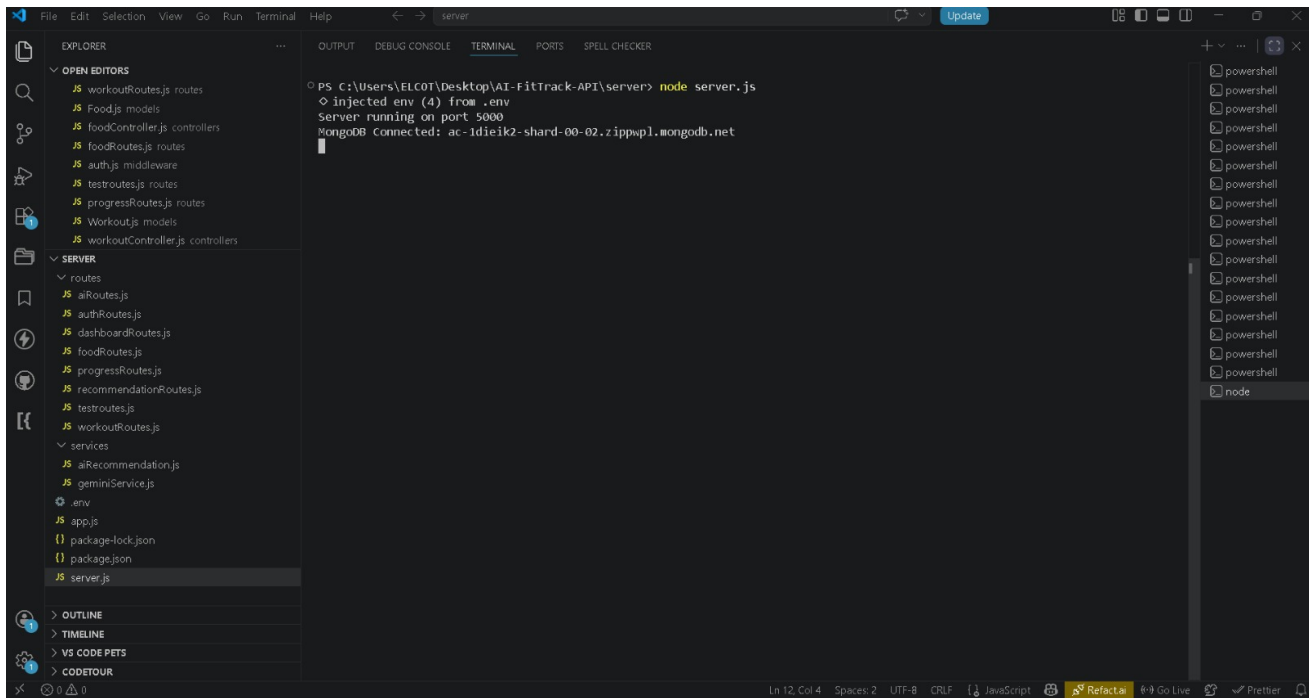
Search **Workouts**

Click "Workouts" to view your workouts.

A full-page view of the running application with Register, Login, Dashboard and all module sections visible.

Supplementary

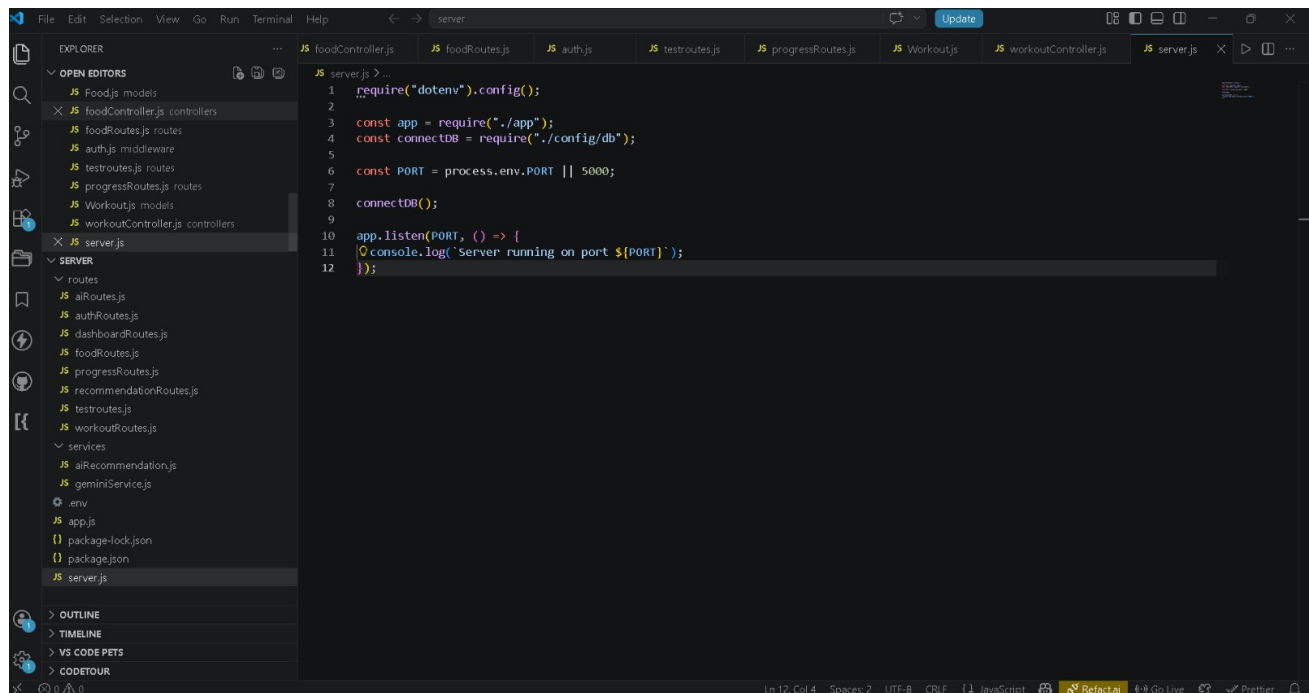
Backend – Server and Database Connection



The screenshot shows the VS Code interface with the Explorer sidebar on the left displaying the project structure. The main editor area shows the terminal output of the command `node server.js`. The output indicates that the server is running on port 5000 and is connected to MongoDB Atlas.

```
PS C:\Users\ELCOT\Desktop\AI-FitTrack-API\server> node server.js
injected env (4) from .env
Server running on port 5000
MongoDB Connected: ac-1dieik2-shard-00-02.zippwpl.mongodb.net
```

A. Backend server running on port 5000 with MongoDB Atlas connected.



The screenshot shows the VS Code interface with the Explorer sidebar on the left displaying the project structure. The main editor area shows the source code of `server.js`, which sets up an Express server and connects to MongoDB Atlas.

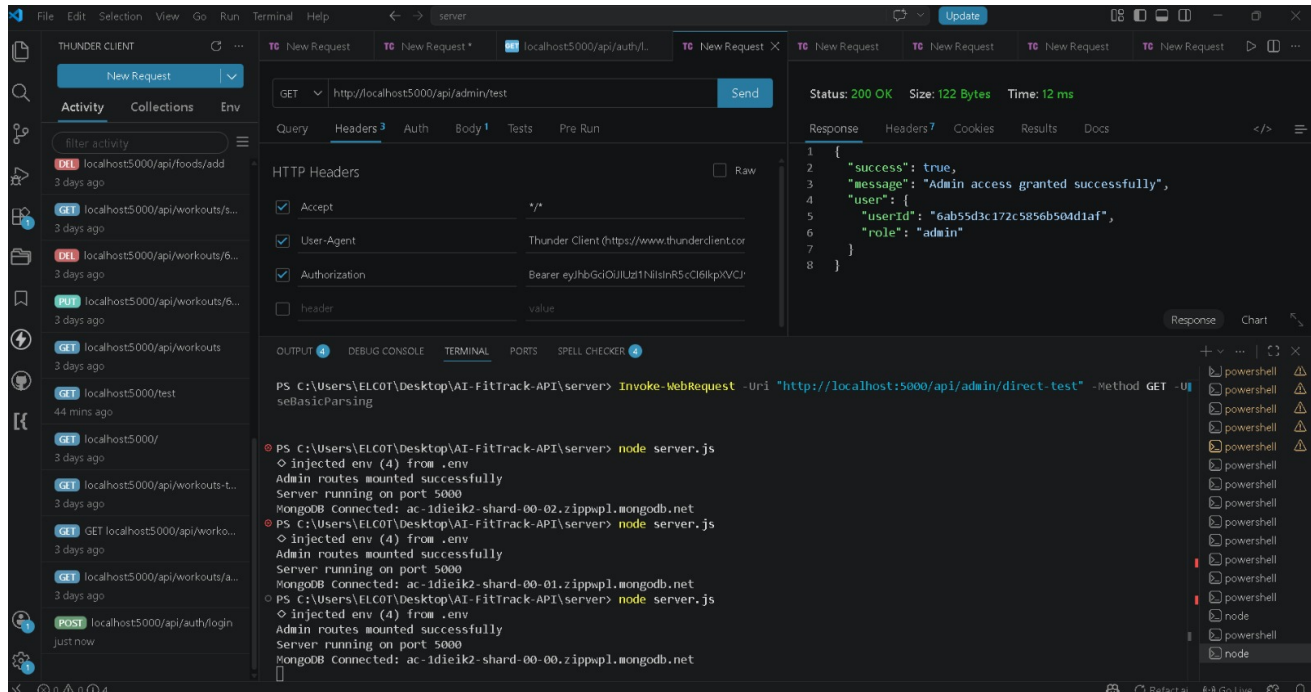
```
server.js
1 require("dotenv").config();
2
3 const app = require("../app");
4 const connectDB = require("../config/db");
5
6 const PORT = process.env.PORT || 5000;
7
8 connectDB();
9
10 app.listen(PORT, () => {
11   console.log("Server running on port ${PORT}");
12 });
```

B. server.js – Express server entry point and database connection call.

17. Additional AI, Vector Search and Admin Feature Screenshots

This section provides direct evidence — Thunder Client requests/responses and terminal output — for the AI Chat, Vector Search and Role-Based Access Control features described in Section 12.

A. Role-Based Access Control – Admin Role Verification



The screenshot displays the Thunder Client interface and a terminal window. The Thunder Client shows a GET request to `http://localhost:5000/api/admin/test` with a status of 200 OK. The response body is:

```

{
  "success": true,
  "message": "Admin access granted successfully",
  "user": {
    "userId": "6ab55d3c172c5856b504d1af",
    "role": "admin"
  }
}

```

The terminal output shows the server running on port 5000 and the request being processed. The output includes the following lines:

```

PS C:\Users\ELCOT\Desktop\AI-FitTrack-API\server> node server.js
◇ injected env (4) from .env
Admin routes mounted successfully
Server running on port 5000
MongoDB Connected: ac-1dieik2-shard-00-02.zippwp1.mongodb.net
PS C:\Users\ELCOT\Desktop\AI-FitTrack-API\server> node server.js
◇ injected env (4) from .env
Admin routes mounted successfully
Server running on port 5000
MongoDB Connected: ac-1dieik2-shard-00-01.zippwp1.mongodb.net
PS C:\Users\ELCOT\Desktop\AI-FitTrack-API\server> node server.js
◇ injected env (4) from .env
Admin routes mounted successfully
Server running on port 5000
MongoDB Connected: ac-1dieik2-shard-00-00.zippwp1.mongodb.net

```

GET `/api/admin/test` returns `{ role: "admin" }` only when the requester's JWT belongs to a user whose role field is admin; the admin middleware rejects the request otherwise.

B. Admin Dashboard API

The screenshot displays the Thunder Client interface. The top bar shows the file explorer with various JavaScript files. The main panel shows a GET request to `http://localhost5000/api/admin/dashboard` with a status of 200 OK. The response body is a JSON object:

```

{
  "success": true,
  "message": "Admin dashboard data fetched successfully",
  "statistics": {
    "totalUsers": 4,
    "totalWorkouts": 7,
    "totalFoods": 4,
    "totalProgress": 8
  }
}

```

The bottom panel shows the terminal output, which includes the following log messages:

```

PS C:\Users\ELCOT\Desktop\AI-FitTrack-API\server> taskkill /f /IM node.exe
SUCCESS: The process "node.exe" with PID 12616 has been terminated.
SUCCESS: The process "node.exe" with PID 28400 has been terminated.
PS C:\Users\ELCOT\Desktop\AI-FitTrack-API\server> node server.js
◇ injected env (4) from .env
Admin routes mounted successfully
Server running on port 5000
MongoDB Connected: ac-1dieik2-shard-00-00.zippap1.mongodb.net
PS C:\Users\ELCOT\Desktop\AI-FitTrack-API\server> node server.js
◇ injected env (4) from .env
Admin routes mounted successfully
Server running on port 5000
MongoDB Connected: ac-1dieik2-shard-00-02.zippap1.mongodb.net
PS C:\Users\ELCOT\Desktop\AI-FitTrack-API\server> node server.js
◇ injected env (4) from .env
Admin routes mounted successfully
Server running on port 5000
MongoDB Connected: ac-1dieik2-shard-00-02.zippap1.mongodb.net

```

GET `/api/admin/dashboard`, protected by the same admin middleware, returns platform-wide statistics (totalUsers, totalWorkouts, totalFoods, totalProgress). The terminal log confirms “Admin routes mounted successfully” at server start.

C. Admin Users API

The screenshot displays the Thunder Client interface. The top bar shows the current request is a GET to `http://localhost5000/api/admin/users` with a status of 200 OK, size of 788 Bytes, and time of 72 ms. The response body is a JSON object:

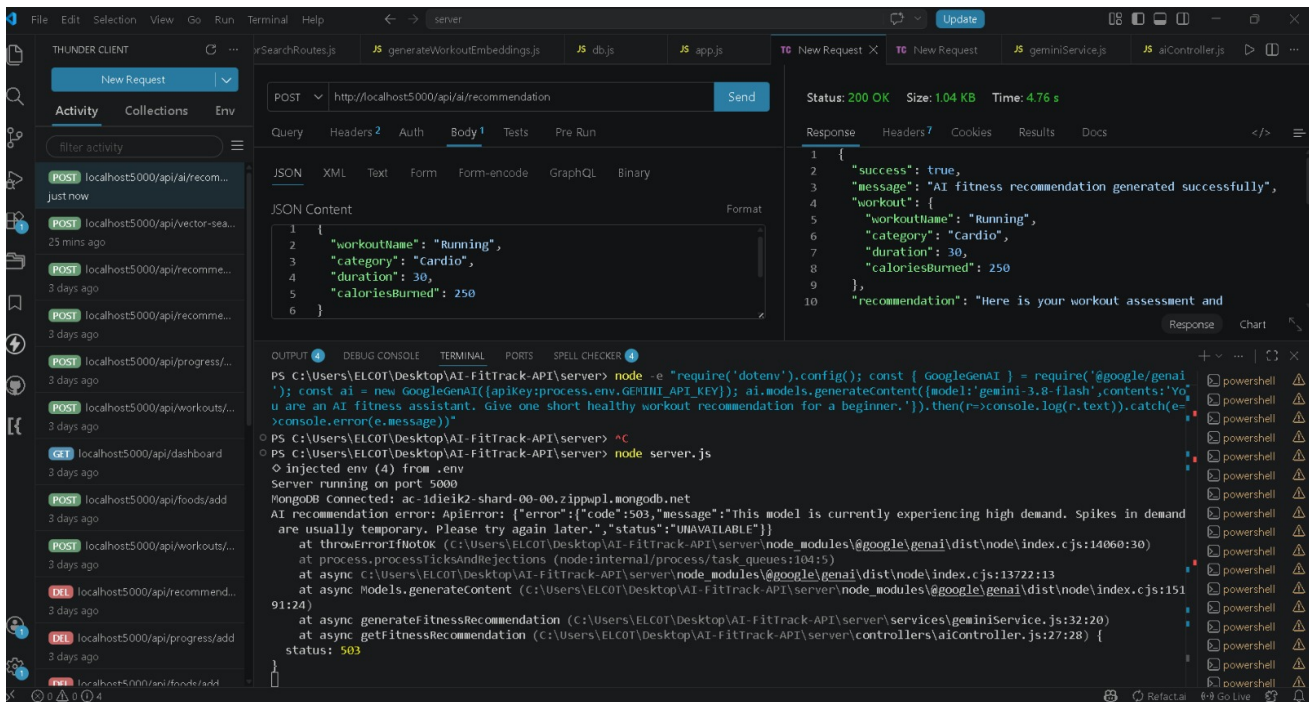
```
{
  "success": true,
  "message": "Users fetched successfully",
  "count": 4,
  "users": [
    {
      "_id": "6ab8f09e67e87e9090006ae0",
      "name": "som",
      "email": "somu33134@gmail.com",
      "createdAt": "2026-09-27T10:31:58.310z",
    }
  ]
}
```

The left sidebar shows a list of recent requests, including GET and POST requests to various API endpoints. The bottom terminal window shows the following output:

```
PS C:\Users\ELC01\Desktop\AI-FitTrack-API\server> taskkill /f /im node.exe
SUCCESS: The process "node.exe" with PID 12616 has been terminated.
SUCCESS: The process "node.exe" with PID 28400 has been terminated.
PS C:\Users\ELC01\Desktop\AI-FitTrack-API\server> node server.js
◇ injected env (4) from .env
Admin routes mounted successfully
Server running on port 5000
MongoDB Connected: ac-1dieik2-shard-00-00.zippap1.mongodb.net
```

GET `/api/admin/users` lists every registered user; this is only reachable with an admin-role token, demonstrating the access-control boundary between the User and Admin roles.

D. Gemini API – Real External Call



The screenshot displays the Thunder Client interface with a REST client request and a terminal window showing the server's execution.

Request Details:

- Method:** POST
- URL:** http://localhost5000/api/ai/recommendation
- Status:** 200 OK
- Size:** 1.04 KB
- Time:** 4.76 s

Request Body (JSON):

```

1 {
2   "workoutName": "Running",
3   "category": "cardio",
4   "duration": 30,
5   "caloriesBurned": 250
6 }
  
```

Response (JSON):

```

1 {
2   "success": true,
3   "message": "AI fitness recommendation generated successfully",
4   "workout": {
5     "workoutName": "Running",
6     "category": "Cardio",
7     "duration": 30,
8     "caloriesBurned": 250
9   },
10  "recommendation": "Here is your workout assessment and
  
```

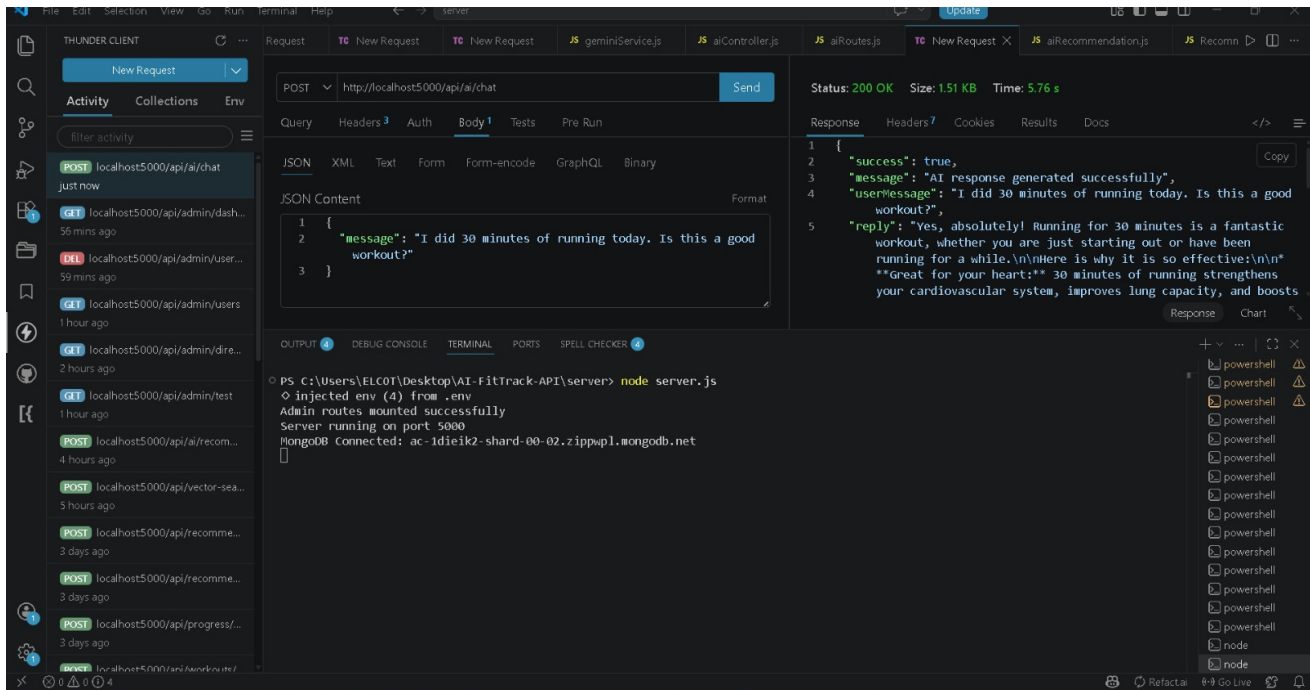
Terminal Output:

```

PS C:\Users\ELCOT\Desktop\AI-FitTrack-API> node -e "require('dotenv').config(); const { GoogleGenAI } = require('@google/genai'); const ai = new GoogleGenAI({apiKey:process.env.GEMINI_API_KEY}); ai.models.generateContent({model:'gemini-3.8-flash',contents:'You are an AI fitness assistant. Give one short healthy workout recommendation for a beginner.'}).then(r=>console.log(r.text)).catch(e=>console.error(e.message))"
PS C:\Users\ELCOT\Desktop\AI-FitTrack-API> ^C
PS C:\Users\ELCOT\Desktop\AI-FitTrack-API> node server.js
  injected env (4) from .env
  Server running on port 5000
MongoDB Connected: ac-1d1e112-shard-00-00.zippapl.mongodb.net
AI recommendation error: ApiError: {"error":{"code":503,"message":"This model is currently experiencing high demand. Spikes in demand are usually temporary, please try again later."},"status":"UNAVAILABLE"}}
    at throwErrorIfNotOk (C:\Users\ELCOT\Desktop\AI-FitTrack-API\server\node_modules\@google\genai\dist\node\index.cjs:14060:30)
    at process.processTicksAndRejections (node:internal/process/task_queues:104:5)
    at async C:\Users\ELCOT\Desktop\AI-FitTrack-API\server\node_modules\@google\genai\dist\node\index.cjs:13722:13
    at async Models.generateContent (C:\Users\ELCOT\Desktop\AI-FitTrack-API\server\node_modules\@google\genai\dist\node\index.cjs:15191:24)
    at async generateFitnessRecommendation (C:\Users\ELCOT\Desktop\AI-FitTrack-API\server\services\geminiService.js:32:20)
    at async getFitnessRecommendation (C:\Users\ELCOT\Desktop\AI-FitTrack-API\server\controllers\aiController.js:27:28) {
  status: 503
}
  
```

POST /api/ai/recommendation returns a Gemini-generated recommendation for a given workout. The terminal also shows a direct node test of the Gemini API (model gemini-3.8-flash) and a transient 503 “model overloaded” error that the service recovered from on the next run — evidence that a real external AI API is being called, not a hard-coded response.

E. AI Chat – Natural-Language Query



The screenshot shows the Thunder Client interface with a POST request to `http://localhost5000/api/ai/chat`. The request body is a JSON object:

```
1 {
2   "message": "I did 30 minutes of running today. Is this a good workout?"
3 }
```

The response status is 200 OK, with a size of 1.51 KB and a time of 5.76 s. The response body is a JSON object:

```
1 {
2   "success": true,
3   "message": "AI response generated successfully",
4   "userMessage": "I did 30 minutes of running today. Is this a good workout?",
5   "reply": "Yes, absolutely! Running for 30 minutes is a fantastic workout, whether you are just starting out or have been running for a while.\n\nHere is why it is so effective:\n\n**Great for your heart:** 30 minutes of running strengthens your cardiovascular system, improves lung capacity, and boosts
```

The terminal output shows the server running on port 5000 and connected to MongoDB.

POST `/api/ai/chat` accepts a free-text message ("I did 30 minutes of running today. Is this a good workout?") and returns a full conversational reply generated by the Gemini API.

F. AI Chat History

The screenshot displays the Thunder Client interface. The top bar shows the current request is a GET to `http://localhost5000/api/ai/chat/history`. The status bar indicates a 200 OK response with a size of 1.64 KB and a time of 424 ms. The response body is a JSON object:

```

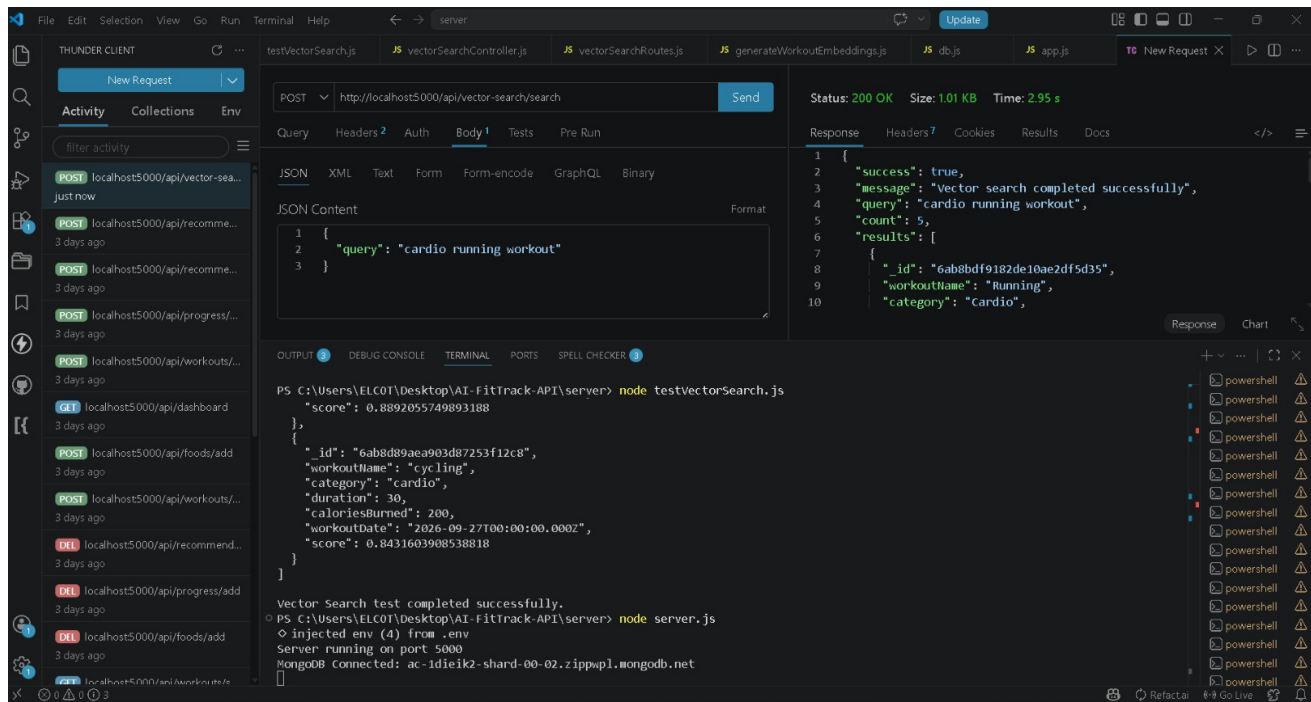
{
  "success": true,
  "message": "Chat history fetched successfully",
  "count": 1,
  "chats": [
    {
      "id": "6abcca43d33046da8fa0e1aa",
      "userId": "6ab55d3c172c5856b504d1af",
      "userMessage": "I did 30 minutes of running today. Is this a good workout?"
    }
  ]
}

```

The left sidebar shows a list of recent requests, including GET and POST requests to various endpoints like `/api/ai/chat/history`, `/api/admin/dash...`, `/api/admin/user...`, `/api/admin/users`, `/api/admin/dire...`, `/api/admin/test`, `/api/ai/recom...`, and `/api/vector-sea...`. The bottom terminal window shows the command `node server.js` being executed, with output indicating that the admin routes are mounted successfully and the server is running on port 5000. It also shows the MongoDB connection status.

GET `/api/ai/chat/history` retrieves the authenticated user's previous chat messages, confirming that each chat exchange is persisted rather than discarded after the response.

G. Vector Search API



The screenshot shows the Thunder Client interface with a POST request to `http://localhost5000/api/vector-search/search` successfully executed. The request body is a JSON object with a `query` of `"cardio running workout"`. The response is a JSON object indicating success and returning a list of workout results.

Request:

```
POST http://localhost5000/api/vector-search/search
{
  "query": "cardio running workout"
}
```

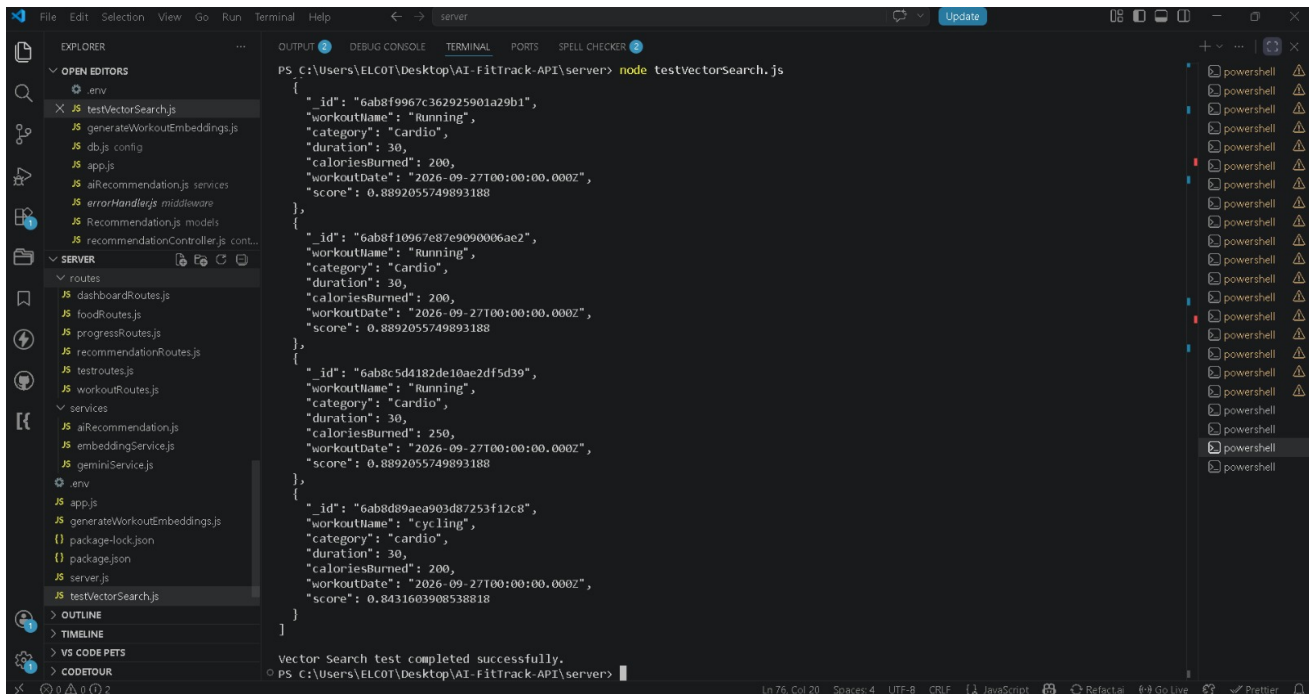
Response:

```
{
  "success": true,
  "message": "Vector search completed successfully",
  "query": "cardio running workout",
  "count": 5,
  "results": [
    {
      "_id": "6ab8bdf9182de10ae2df5d35",
      "workoutName": "Running",
      "category": "Cardio",
      "score": 0.8892055749893188
    }
  ]
}
```

The terminal output shows the command `node testVectorSearch.js` being executed, which outputs the search results and confirms the test was successful.

POST `/api/vector-search/search` with the query `"cardio running workout"` returns workouts ranked by cosine-similarity score against pre-generated embedding vectors, rather than an exact text match.

H. testVectorSearch.js – Standalone Verification



```

PS C:\Users\ELCOT\Desktop\AI-FitTrack-API\server> node testVectorSearch.js

[
  {
    "_id": "6ab8f9967c362925901a29b1",
    "workoutName": "Running",
    "category": "Cardio",
    "duration": 30,
    "caloriesBurned": 200,
    "workoutDate": "2026-09-27T00:00:00.000Z",
    "score": 0.8892055749893188
  },
  {
    "_id": "6ab8f10967e87e9090006ae2",
    "workoutName": "Running",
    "category": "Cardio",
    "duration": 30,
    "caloriesBurned": 200,
    "workoutDate": "2026-09-27T00:00:00.000Z",
    "score": 0.8892055749893188
  },
  {
    "_id": "6ab8c5d4182de10ae2df5d39",
    "workoutName": "Running",
    "category": "Cardio",
    "duration": 30,
    "caloriesBurned": 250,
    "workoutDate": "2026-09-27T00:00:00.000Z",
    "score": 0.8892055749893188
  },
  {
    "_id": "6ab8d89aea903d87253f12c8",
    "workoutName": "cycling",
    "category": "cardio",
    "duration": 30,
    "caloriesBurned": 200,
    "workoutDate": "2026-09-27T00:00:00.000Z",
    "score": 0.8431603908538818
  }
]

Vector Search test completed successfully.
PS C:\Users\ELCOT\Desktop\AI-FitTrack-API\server>
  
```

A standalone script queries the stored workout embeddings directly and prints each match with its similarity score, used to verify the vector search logic independently of the HTTP API.

18. Demo Video and Source Code Links

Demo Video Link

[To be added by the team – insert the demo video URL here before submission]

Source Code Drive / GitHub Link

<https://github.com/SOMUMP/AI-FitTrack.git>

19. Mandatory Functionalities Checklist

The table below lists the program's mandatory functionality items that are implemented in AI FitTrack, with the evidence location for each.

Mandatory Item	Status	Notes / Evidence
MVC Architecture	Implemented	models/ controllers/ routes/ services/ middleware/
JWT Authentication	Implemented	jsonwebtoken + bcryptjs
REST API (CRUD)	Implemented	Progress, workout, food modules
Real AI/LLM API Integration	Implemented	Google Gemini API – Section 17-D
Prompt Engineering	Implemented	BMI/activity + chat prompts – Section 12
Role-Based Access Control	Implemented	Admin middleware + role field – Section 17-A/B/C
Semantic Search	Implemented	Embedding-based workout search – Section 17-G
MongoDB Atlas Vector Search	Implemented	Workout embeddings + similarity ranking – Section 17-G/H
AI Chat	Implemented	POST /api/ai/chat – Section 17-E
Chat History	Implemented	GET /api/ai/chat/history – Section 17-F
Natural-Language Query Flow	Implemented	Free-text chat input/output – Section 17-E
Embeddings	Implemented	generateWorkoutEmbeddings.js – Section 17-G/H

20. Conclusion

AI FitTrack combines a browser-based frontend, a JWT-secured Express/MongoDB backend and a Google Gemini AI layer into a single fitness management system. It demonstrates practical use of the MVC pattern, REST API design, MongoDB Atlas, role-based access control, and integration with a generative-AI service across three real features — personalized recommendations, a conversational AI chat, and embedding-based vector search — while keeping the architecture modular enough to extend with the features listed below.

21. Future Enhancements

- MongoDB Atlas Search (text/autocomplete index) alongside the existing vector search
- Document upload and Document Q&A; (retrieval-augmented generation over user-uploaded files)
- Charts for weight and workout progress over time
- More detailed nutrition analytics
- Workout reminders and notifications
- Wearable-device integration
- Exporting progress reports (PDF/CSV)
- Cloud deployment of frontend and backend
- Additional automated testing (unit + integration)